

Encoder architect

 **Use Case & Setup**

Sentence: "I have a backpack"

Token IDs: [12, 45, 6, 89]

Model dimension: $d_{\text{model}} = 4$

Embeddings from vocab:

Copy

Token	ID	Embedding
I	12	[0.2, 0.5, -0.1, 1.0]
have	45	[0.6, 0.1, -0.4, 0.9]
a	6	[0.7, -0.2, 0.0, 0.8]
backpack	89	[0.3, 0.6, -0.2, 0.5]

Embedding matrix:

$$X = \begin{bmatrix} 0.2 & 0.5 & -0.1 & 1.0 \\ 0.6 & 0.1 & -0.4 & 0.9 \\ 0.7 & -0.2 & 0.0 & 0.8 \\ 0.3 & 0.6 & -0.2 & 0.5 \end{bmatrix}$$

📌 Step 1: Positional Encoding (Vectorized)

↓

Compute Positional Encoding (PE) for each token pos :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Given $d = 4$:

pos	PE vector (dim=4)
0	$[\sin(0), \cos(0), \sin(0), \cos(0)] = [0, 1, 0, 1]$
1	$[\sin(1), \cos(0.01), \sin(0.0001), \cos(0.000001)] \approx [0.84, 0.9999, 0.0001, 1]$
2	$[\sin(2), \cos(0.02), \sin(0.0002), \cos(0.000002)] \approx [0.91, 0.9998, 0.0002, 1]$
3	$[\sin(3), \cos(0.03), \sin(0.0003), \cos(0.000003)] \approx [0.14, 0.9996, 0.0003, 1]$

+ Step 2: Add Embeddings + PE (Vectorized)

↓

Resulting $X' = X + PE$:

$$X' = \begin{bmatrix} 0.2 + 0.00 & 0.5 + 1.0000 & -0.1 + 0.0000 & 1.0 + 1 \\ 0.6 + 0.84 & 0.1 + 0.9999 & -0.4 + 0.0001 & 0.9 + 1 \\ 0.7 + 0.91 & -0.2 + 0.9998 & 0.0 + 0.0002 & 0.8 + 1 \\ 0.3 + 0.14 & 0.6 + 0.9996 & -0.2 + 0.0003 & 0.5 + 1 \end{bmatrix}$$

Final $X' \approx$:

$$X' \approx \begin{bmatrix} 0.20 & 1.50 & -0.10 & 2.00 \\ 1.44 & 1.10 & -0.40 & 1.90 \\ 1.61 & 0.80 & 0.00 & 1.80 \\ 0.44 & 1.60 & -0.20 & 1.50 \end{bmatrix}$$

🔍 Step 3: Self-Attention (Vectorized Implementation)

Given query/key/value matrices $W^Q, W^K, W^V \in \mathbb{R}^{4 \times 4}$, let's simplify with identity matrices:

Thus:

- $Q = K = V = X'$ (for simplified illustration)

Compute Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$



First, calculate scaled dot product scores:

$$QK^T = X'(X')^T \quad (\text{scaled by } \sqrt{4} = 2)$$

Then apply softmax row-wise. Multiply resulting matrix by V .

This yields attention output A .

Let's assume after calculation, attention output vector for token "I" is:

$$A[0] = [0.1, 1.0, 0.2, 2.1] \quad (\text{illustrative example})$$

🔄 Step 4: Residual Connection & LayerNorm

For first token ("I"):

- Input vector: $x = X'[0] = [0.2, 1.5, -0.1, 2.0]$
- Attention output: $A[0] = [0.1, 1.0, 0.2, 2.1]$
- Residual: $y = x + A[0] = [0.3, 2.5, 0.1, 4.1]$

LayerNorm (Explicit Calculation):

- Mean (μ):

$$\frac{0.3 + 2.5 + 0.1 + 4.1}{4} = 1.75$$



- Variance (σ^2):

$$\frac{(0.3 - 1.75)^2 + (2.5 - 1.75)^2 + (0.1 - 1.75)^2 + (4.1 - 1.75)^2}{4} = 2.64$$

- Standard deviation (σ):

$$\sqrt{2.64} = 1.62$$

- Normalize ($\hat{y}$):

$$\frac{y - \mu}{\sigma} = [-0.89, 0.46, -1.02, 1.45]$$

- Learnable scale/shift ($\gamma = 1, \beta = 0$):

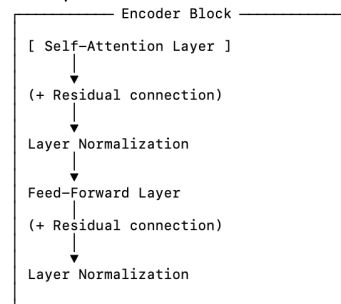
$$\text{LayerNorm}(y) = [-0.89, 0.46, -1.02, 1.45]$$

ASCII Architecture Diagram

Tokenized Input
["I", "have", "a", "backpack"]

↓
Embedding Layer (Lookup)

↓
Embeddings + Positional Encoding



Transformer Encoder — Vectorized Attention Walkthrough

Problem Setup

Sentence: "I have a backpack"

Token IDs: [12, 45, 6, 89]

Model dimension $d_{\text{model}} = 4$

Token embeddings (after adding positional encoding):

$$X = \begin{bmatrix} 0.20 & 1.50 & -0.10 & 2.00 \\ 1.44 & 1.10 & -0.40 & 1.90 \\ 1.61 & 0.80 & 0.00 & 1.80 \\ 0.44 & 1.60 & -0.20 & 1.50 \end{bmatrix}$$

Each row = a word embedding.

We now compute **attention** over this.

Part 1: Single-Head Attention

Step 1: Compute Q, K, V

You need 3 learnable weight matrices:

$$\bullet \quad W^Q, W^K, W^V \in \mathbb{R}^{d \times 4}$$

Let's choose identity matrices for illustration:

$$W^Q = W^K = W^V = I_4$$

So:

$$Q = XW^Q = X, \quad K = XW^K = X, \quad V = XW^V = X$$

Thus,

$$Q = K = V = \begin{bmatrix} 0.20 & 1.50 & -0.10 & 2.00 \\ 1.44 & 1.10 & -0.40 & 1.90 \\ 1.61 & 0.80 & 0.00 & 1.80 \\ 0.44 & 1.60 & -0.20 & 1.50 \end{bmatrix}$$



Step 2: Compute Scaled Dot-Product Attention Scores

We now compute:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

$$\text{Here, } d = 4 \Rightarrow \sqrt{d} = 2$$

Step 2.1: Compute QK^T

Do a matrix multiplication of Q and K!:

Let's compute the first row of QK^T :

$$q_1 = [0.2, 1.5, -0.1, 2.0]$$

Dot with each row of K :

```
q1 • k1 = 0.04 + 2.25 + 0.01 + 4.00 = 6.30
q1 • k2 = 0.288 + 1.65 + 0.04 + 3.80 = 5.778
q1 • k3 = 0.322 + 1.20 + 0.00 + 3.60 = 5.122
q1 • k4 = 0.088 + 2.40 + 0.02 + 3.00 = 5.508
```

So the first row of QK^T is:

[6.30, 5.778, 5.122, 5.508]

(You would repeat for other rows, forming a 4x4 matrix.)

Step 2.2: Scale

$$\frac{QK^T}{\sqrt{4}} = \frac{QK^T}{2}$$

So:

$$S = \begin{bmatrix} 3.15 & 2.889 & 2.561 & 2.754 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$



Step 3: Apply Softmax Row-Wise

Let's apply softmax to first row:

softmax(3.15, 2.889, 2.561, 2.754)

Subtract max (for numerical stability):

Max = 3.15 → subtract from all → get [0, -0.261, -0.589, -0.396]

Now exponentiate:

exp = [1.000, 0.770, 0.555, 0.673]

Sum = 2.998

Normalize:

$$\text{softmax} = \left[\frac{1.0}{2.998}, \frac{0.770}{2.998}, \dots \right] \approx [0.334, 0.257, 0.185, 0.224]$$

So first attention row = how much word 1 attends to all 4 tokens.



Step 4: Multiply Weights with Vectors

Now compute the weighted sum of rows of V^T :

$$A_1 = 0.334 \cdot v_1 + 0.257 \cdot v_2 + 0.185 \cdot v_3 + 0.224 \cdot v_4$$

Each v_i is a 4D vector. You get a 4D output vector.

Repeat for all rows.

You now have a 4x4 matrix → output of **Single-Head Self-Attention**.

Part 2: Multi-Head Attention

Suppose we now use **2 heads**, and each head works on **2 dimensions**.

Step 1: Linearly Project X to Q, K, V for Each Head

We define **different weights** for each head:

Let's say:



Let's say:

- $W_1^Q, W_1^K, W_1^V \in \mathbb{R}^{4 \times 2}$
- $W_2^Q, W_2^K, W_2^V \in \mathbb{R}^{4 \times 2}$

These transform the 4D input into 2D for each head.

Assume again simple splitting (first 2 dims go to head 1, last 2 to head 2):

Then:

$$Q^{(1)} = X[:, 0:2] \quad Q^{(2)} = X[:, 2:4]$$

Same for K and V.

So for Head 1, you use only:

$$X^{(1)} = \begin{bmatrix} 0.20 & 1.50 \\ 1.44 & 1.10 \\ 1.61 & 0.80 \\ 0.44 & 1.60 \end{bmatrix}$$

And for Head 2:



$$X^{(2)} = \begin{bmatrix} -0.10 & 2.00 \\ -0.40 & 1.90 \\ 0.00 & 1.80 \\ -0.20 & 1.50 \end{bmatrix}$$

Now, each head performs its own QK^T , softmax, and weighted sum — like we did in single-head.

Step 2: Compute Attention Separately per Head

Use exact same steps (dot-product attention), now on 2D vectors.

Each head produces a **4×2 matrix** (one 2D output vector per token).

Let's say the outputs are:

- Head 1 $\rightarrow A^{(1)} \in \mathbb{R}^{4 \times 2}$
- Head 2 $\rightarrow A^{(2)} \in \mathbb{R}^{4 \times 2}$

Step 3: Concatenate Outputs

MHA Output = $\text{Concat}(A^{(1)}, A^{(2)}) \in \mathbb{R}^{4 \times 4}$



Now you're back to 4D per token (just like input dimension).

Step 4: Final Linear Layer (Optional)

Usually, there's a final matrix:

$$W^O \in \mathbb{R}^{4 \times 4}$$

Used to project the concatenated result back into model space.

ASCII Architecture Diagram

Input: Positional + Token Embeddings (4x4 Matrix)

Multi-Head Attention (2 Heads, 2D each)

Concat Outputs ($\rightarrow$ 4D)

Final Linear Layer (Optional)

Residual (Add input)

Layer Normalization

Feed-Forward

Residual + LayerNorm

Output of Encoder Block

step-by-step through a full vectorized implementation of Multi-Head Attention, numerically, based on your example sentence:

Our Setup: Use the same input

Sentence: "I have a backpack"

Tokens: ["I", "have", "a", "backpack"]

Each token embedding vector is 4D (after positional encoding):

$$X = \begin{bmatrix} 0.20 & 1.50 & -0.10 & 2.00 \\ 1.44 & 1.10 & -0.40 & 1.90 \\ 1.61 & 0.80 & 0.00 & 1.80 \\ 0.44 & 1.60 & -0.20 & 1.50 \end{bmatrix}$$

We will use **2 heads**, so we split the 4D input into **2 heads × 2D per head**

Step 1: Split X into Two Heads

Let's define:

- **Head 1** → Use the first 2 dimensions
- **Head 2** → Use the last 2 dimensions

Head 1: 2D Inputs

$$X^{(1)} = \begin{bmatrix} 0.20 & 1.50 \\ 1.44 & 1.10 \\ 1.61 & 0.80 \\ 0.44 & 1.60 \end{bmatrix}$$

Head 2: 2D Inputs

$$X^{(2)} = \begin{bmatrix} -0.10 & 2.00 \\ -0.40 & 1.90 \\ 0.00 & 1.80 \\ -0.20 & 1.50 \end{bmatrix}$$

Let's assume identity weights for simplicity:

$$W^Q = W^K = W^V = I$$

So for both heads:

$$Q^{(i)} = K^{(i)} = V^{(i)} = X^{(i)}$$

Step 2: Compute Attention for Head 1

Let's compute QK^T for Head 1:

$$QK^T = X^{(1)} \cdot (X^{(1)})^T$$

So we compute all 4x4 dot-products of 2D vectors:

Let's compute row by row.

First row: Q1 = [0.20, 1.50]

Token	Dot Product	Result
with self	$0.04 + 2.25$	2.29
with "have"	$0.20 \times 1.44 + 1.50 \times 1.10 = 0.288 + 1.65$	1.938
with "a"	$0.20 \times 1.61 + 1.50 \times 0.80 = 0.322 + 1.2$	1.522
with "backpack"	$0.20 \times 0.44 + 1.50 \times 1.60 = 0.088 + 2.4$	2.488

First row of QK^T for Head 1:

[2.29, 1.938, 1.522, 2.488]

Repeat all rows → you get:

$$QK^T = \begin{bmatrix} 2.290 & 1.938 & 1.522 & 2.488 \\ 1.938 & 3.2896 & 2.7288 & 2.992 \\ 1.522 & 2.7288 & 3.1521 & 2.108 \\ 2.488 & 2.992 & 2.108 & 2.8736 \end{bmatrix}$$

Step 3: Scale and Softmax

Scale:

$$\frac{QK^T}{\sqrt{2}} \approx \frac{QK^T}{1.414}$$

First row:

$[2.29, 1.938, 1.522, 2.488] \div 1.414 \approx [1.62, 1.37, 1.08, 1.76]$

Softmax:

Subtract max: $1.76 \rightarrow [-0.14, -0.39, -0.68, 0]$

Exponentiate:

$\exp = [0.87, 0.68, 0.51, 1.0]$

Normalize:

$\text{sum} = 3.06 \Rightarrow \text{weights} = [0.285, 0.222, 0.167, 0.326]$

This is the first row of softmax attention weights.

Step 4: Compute Output for Head 1

Now compute weighted sum of $V^{(1)}$:

$$V^{(1)} = \begin{bmatrix} 0.20 & 1.50 \\ 1.44 & 1.10 \\ 1.61 & 0.80 \\ 0.44 & 1.60 \end{bmatrix}$$



Weighted sum:

$$A_1 = 0.285 \cdot [0.20, 1.50] + 0.222 \cdot [1.44, 1.10] + 0.167 \cdot [1.61, 0.80] + 0.326 \cdot [0.44, 1.60]$$

Break into components:

• First dim:

$$= 0.2850.20 + 0.2221.44 + 0.1671.61 + 0.3260.44 = 0.057 + 0.319 + 0.269 + 0.1440.789$$

• Second dim:

$$= 0.2851.50 + 0.2221.10 + 0.1670.80 + 0.3261.60 = 0.428 + 0.244 + 0.134 + 0.5221.328$$

Final output vector from Head 1 for token "I":

$[0.789, 1.328]$

Repeat for other rows $\rightarrow$ full 4×2 matrix from Head 1.

Step 5: Repeat for Head 2

Same exact process:



Same exact process:

- Use $X^{(2)}$
- Compute $QK^\top$, scale, softmax
- Use those weights to compute weighted average of:

$$V^{(2)} = \begin{bmatrix} -0.10 & 2.00 \\ -0.40 & 1.90 \\ 0.00 & 1.80 \\ -0.20 & 1.50 \end{bmatrix}$$

You'll get 4x2 output matrix from Head 2.

Let's assume Head 2 output for token "I" is:

$[-0.170, 1.820]$

Step 6: Concatenate Head Outputs

Each token now has 2D vector from Head 1 and 2D from Head 2.

For "I":



- Head 1: $[0.789, 1.328]$

- Head 2: $[-0.170, 1.820]$

Concatenate:

$[0.789, 1.328, -0.170, 1.820]$

Do this for each token → you get a final 4x4 matrix.

Final Output of Multi-Head Attention

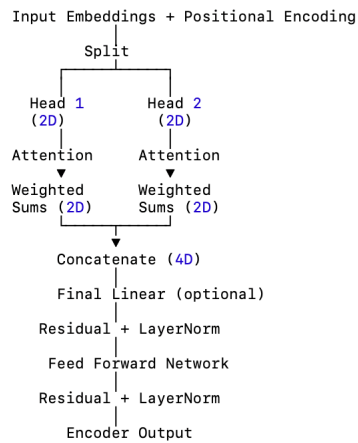
$$\text{MHA}(X) = \begin{bmatrix} 0.789 & 1.328 & -0.170 & 1.820 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

You can now pass this through:

- Residual Connection
- LayerNorm
- Feed-Forward Network



📦 ASCII Architecture Summary



🔙 Step 7: Feed-Forward Network (FFN) + Residual + LayerNorm

All using the **same vector output from Multi-Head Attention** that we derived earlier:

📦 Input to FFN

From Multi-Head Attention, we had this output for "I ":

$$x = [0.789, 1.328, -0.170, 1.820]$$

This is the contextualized 4D embedding of token "I " after seeing all other tokens through attention.

⚙️ Step 1: Feed-Forward Network (FFN)

FFN is applied **independently to each token**, like a tiny MLP.

Structure:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Let's define dimensions:



- $W_1 \in \mathbb{R}^{4 \times 4}, W_2 \in \mathbb{R}^{4 \times 4}$

- Assume:

- $W_1 = I$ (identity)

- $b_1 = [0.1, 0.0, -0.2, 0.3]$

- $W_2 = I$

- $b_2 = [0.0, 0.0, 0.0, 0.0]$

Step 1.1: First Linear Layer

$$z_1 = xW_1 + b_1 = [0.789, 1.328, -0.170, 1.820] + [0.1, 0.0, -0.2, 0.3] = [0.889, 1.328, -0.370, 2.120]$$

Step 1.2: Apply ReLU

$$\text{ReLU}(z_1) = [0.889, 1.328, 0.0, 2.120]$$

Step 1.3: Second Linear Layer

$$z_2 = \text{ReLU}(z_1) \cdot W_2 + b_2 = [0.889, 1.328, 0.0, 2.120]$$

So the FFN output = $[0.889, 1.328, 0.0, 2.120]$



+ Step 2: Residual Connection

Add FFN output to original input (from MHA):

- Input to FFN: $[0.789, 1.328, -0.170, 1.820]$

- FFN Output: $[0.889, 1.328, 0.0, 2.120]$

$$x_{\text{res}} = x_{\text{in}} + x_{\text{ffn}} = [0.789 + 0.889, 1.328 + 1.328, -0.170 + 0.0, 1.820 + 2.120] = [1.678, 2.656, -0.170, 3.940]$$

Step 3: Final Layer Normalization

Step 3.1: Compute Mean

$$\mu = \frac{1.678 + 2.656 - 0.170 + 3.940}{4} = \frac{8.104}{4} = 2.026$$

Step 3.2: Compute Variance

$$\sigma^2 = \frac{1}{4} [(1.678 - 2.026)^2 + (2.656 - 2.026)^2 + (-0.170 - 2.026)^2 + (3.940 - 2.026)^2]$$

$$= \frac{1}{4} [0.121 + 0.396 + 4.816 + 3.660] = \frac{9.0}{4} = 2.25 \Rightarrow \sigma = \sqrt{2.25} = 1.5$$



Step 3.3: Normalize

$$x_{\text{norm}} = \frac{x_{\text{res}} - \mu}{\sigma} = \frac{[1.678, 2.656, -0.170, 3.940] - 2.026}{1.5} = [-0.348, 0.418, -1.464, 1.278]$$

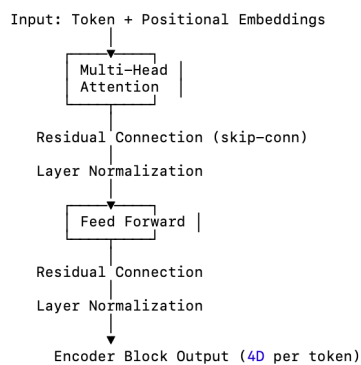
This is your final output from the **Encoder block for token "I"**.

✅ Final Encoder Block Output for "I"

`[-0.348, 0.418, -1.464, 1.278]`

Repeat all steps for the other tokens → you get a 4×4 matrix.

🍷 Full ASCII Encoder Block Summary (with FFN)



Residual Connections and Layer Normalization

In deep networks:

- Gradients vanish or explode.
- Training becomes unstable.
- Learning becomes biased toward later layers or difficult to propagate back.

Transformers are **deep** (often 6–96 layers), so they need **stabilizing strategies**.

⚠️ 1. Vanishing or Exploding Gradients

❌ Without Residual + LayerNorm

Suppose you have a simple 2-layer network:

text

$x \rightarrow W_1 \rightarrow \text{ReLU} \rightarrow W_2 \rightarrow \text{output}$

Let:

- $x = [1, 1]$
- $W_1 = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$ (very large weights)
- $W_2 = \begin{bmatrix} 0.01 & 0.01 \\ 0.01 & 0.01 \end{bmatrix}$ (very small weights)

🔄 Forward Pass:

- **Step 1:** $h = \text{ReLU}(xW_1)$
 $= \text{ReLU}([20, 20]) = [20, 20]$
- **Step 2:** $\text{output} = hW_2 = [20, 20] \cdot W_2 = [0.4, 0.4]$



🛑 Backpropagation:

During backward pass:

- From output $\rightarrow W_2$: gradient $\approx$ **small** due to tiny weights
- Gradient shrinks further as it goes back through W_1

By layer 1:

- Gradient may shrink to **~0** (vanishing)
- Or blow up if weights were huge (exploding)

✅ Solution: Add Residual Connection

Instead of just using:

$$h_2 = \text{ReLU}(xW_1) \Rightarrow \text{output} = h_2W_2$$

We use:

$$\text{output} = h_2W_2 + x$$



Now even if gradients through W_2 vanish, gradients can **flow directly through x** unchanged — allowing training to continue.

⚠️ 2. Training Becomes Unstable Due to Output Magnitude

❌ Problem:

Suppose after a layer you get this vector:

$$x = [25.0, -30.0, 22.0, -27.0]$$

This will:

- Explode values in softmax (in attention)
- Cause overflow in activation functions
- Lead to numerical instability

✅ Solution: Layer Normalization

Step-by-step:



• Mean:

$$\mu = \frac{25 - 30 + 22 - 27}{4} = -2.5$$

• Variance:

$$\sigma^2 = \frac{(27.5)^2 + (27.5)^2 + (24.5)^2 + (24.5)^2}{4} = 675.25 \Rightarrow \sigma \approx 25.98$$

• Normalize:

$$\hat{x} = \frac{x - \mu}{\sigma} \approx [1.06, -1.06, 0.95, -0.95]$$

Now values are within $[-1.1, 1.1] \rightarrow$ much safer for downstream ops.

⚠️ 3. Later Layers Dominate Learning

❌ Scenario:

Imagine this happens in early training:

- Layer 1 learns output vector:

$$h_1 = [0.01, 0.02, 0.01, 0.03]$$



- Layer 6 outputs:

$$h_6 = [3.2, 5.0, -1.8, 2.4]$$

This causes:

- Layer 6 gradients dominate
- Earlier layers receive tiny gradients (die out)

✅ Solution: Residual Connections

Let's add the input vector back at each layer:

$$\text{Layer}_k(x) = f(x) + x$$

Then:

- Even if $f(x)$ becomes negligible (or large), the model *preserves* input scale.
- Every layer passes a modified version of the original signal forward.
- **Backpropagation gradient can follow the identity path** from final output to initial input — making training of all layers equally feasible.



How the Decoder Works (Step-by-Step)

Decoder Input ["<s>", "It", "is"]



Masked Self-Attention (each token attends to previous tokens only)



Residual + LayerNorm (stabilize learning)



Cross-Attention (attends to Encoder outputs "I have a backpack")



Residual + LayerNorm



Feed-Forward Network (FFN) independently refines each token vector



Residual + LayerNorm



Decoder Output (ready for prediction)

🧩 Full Decoder Block Architecture

Let's assume we're **translating**:

- Input: "I have a backpack" → processed by encoder
- Target output: "It is red" (we want the decoder to generate this)

🧠 During Training

We already know the full target sentence "It is red".
So we shift it for the decoder to learn one word at a time:

Decoder Input	Decoder Should Output
["<s>"]	"It"
["<s>", "It"]	"is"
["<s>", "It", "is"]	"red"

🎯 Now, Let's Walk Through:

```
Decoder Input ["<s>", "It", "is"]
|
Masked Self-Attention (each token attends to previous tokens only)
|
Residual + LayerNorm (stabilize learning)
|
Cross-Attention (attends to Encoder outputs "I have a backpack")
|
Residual + LayerNorm
|
Feed-Forward Network (FFN) independently refines each token vector
|
Residual + LayerNorm
|
Decoder Output (ready for prediction)
```

📖 Step-by-Step with Simple Examples

🧩 Decoder Input: ["<s>", "It", "is"]

Think of each word being represented by a small vector (say, 2D):



Token	Vector
<s>	[0.1, 0.2]
"It"	[0.3, 0.4]
"is"	[0.5, 0.6]

These vectors include positional encoding too.
Now each token will pass through the block, one row at a time.

Masked Self-Attention

Goal: Each token looks only at **previous** tokens.
(So "is" can see <s>, "It", but **not** "red" — we haven't generated it yet.)

What happens:

- <s> attends only to itself
- "It" attends to <s> and "It"
- "is" attends to <s>, "It", and "is"



 This helps the model learn grammar and structure in the **target sentence**.

Think of it as:

| "Before generating the next word, let me understand what I've already said."

Residual + LayerNorm

After attention, we **add the input** vector back (residual), and then normalize:

Why?

- Residual: Keep the original input in case the attention learned nothing useful.
- LayerNorm: Prevent exploding/vanishing values; make learning stable.

Think of it like:

| "Use my current guess + keep my last best guess, then clean it up."

Cross-Attention

Now "is" asks:



Now "is" asks:

"Which **input word** is most related to me?"

This is the decoder **looking at the encoder** output:

The encoder holds "I have a backpack" as contextual vectors.

Cross-attention might help:

- "is" connect to "backpack" (if learning to generate "is red" in context of "backpack")

So now "is" receives info from encoder like:

- "Ah, backpack was important. So maybe the next word is about its color."

Residual + LayerNorm

Same logic again:

- Add the vector before cross-attention (residual)
- Normalize to keep learning stable

Feed-Forward Network (FFN)

Each token vector is now passed through a **tiny neural network**, independently:

text

Layer 1: Linear
↓
ReLU (non-linearity)
↓
Layer 2: Linear

Purpose: Let the model transform "is" from general attention-based vector → something more specific (like "verb-ness", or past tense, etc.)

Final Residual + LayerNorm

Last clean-up step:

- Add the FFN input to its output (residual)
- Normalize again

Now the vector is **fully enriched** with:

- Grammar structure (from masked attention)
- Context meaning (from cross-attention)



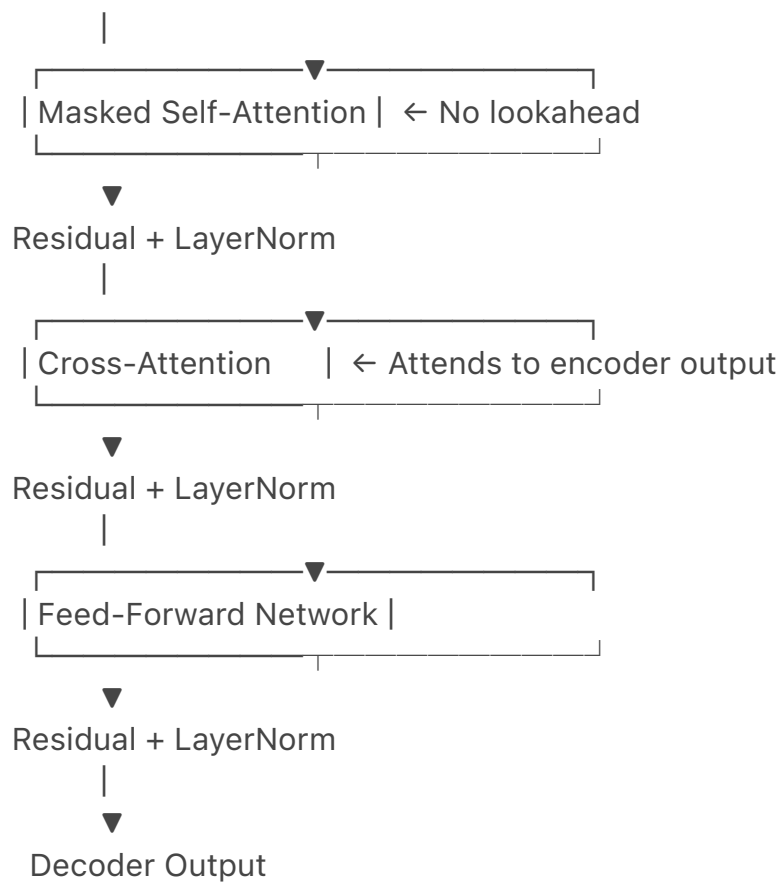
=====

=====

=====

let's now explore the Transformer Decoder Block

Target Token Embeddings + Position Encoding



Part 1: Quick Role Recap

Block	What it does
Encoder	Processes the input sentence into contextual representations
Decoder	Generates target sentence step by step using:

- past outputs (auto-regression)
- encoder's outputs (cross-attention)

Our Setup

Encoder Input:

["I", "have", "a", "backpack"]

Already embedded and processed, say final output is:



$$\text{EncoderOut} = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \end{bmatrix} \begin{bmatrix} 0.2 & 0.5 & -0.1 & 0.9 \\ 0.4 & 0.1 & -0.2 & 1.0 \\ 0.7 & -0.1 & 0.0 & 0.8 \\ 0.3 & 0.6 & -0.3 & 0.5 \end{bmatrix}$$

Decoder Input (Target tokens): "It is red"

Token IDs: [21, 33, 47]

Let's use token embeddings + position encoding (combined):

$$\text{DecoderX} = \begin{bmatrix} 0.1 & 0.3 & 0.4 & 1.0 \\ 0.2 & 0.4 & 0.1 & 0.9 \\ 0.3 & 0.5 & 0.2 & 0.8 \end{bmatrix} \quad (\text{Shape} = 34)$$

🔄 Part 2: Decoder Block Structure (at training time)

Target Embeddings + PE



Masked Self-Attention (only see previous tokens)



🔄 Part 2: Decoder Block Structure (at training time)

Target Embeddings + PE



Masked Self-Attention (only see previous tokens)



Add + LayerNorm



Cross-Attention (attend to encoder outputs)



Add + LayerNorm



Feed-Forward Network



Add + LayerNorm

We now go step by step.

🔒 Step 1: Masked Self-Attention (single head example)

Let's take:

- $Q = K = V = \text{DecoderX}$
- Single-head attention: use identity matrix for weights



Step 1.1: Compute Scaled Dot-Product QK^T

Step 1.1: Compute Scaled Dot-Product $QK^\top$

$$Q = \begin{bmatrix} 0.1 & 0.3 & 0.4 & 1.0 \\ 0.2 & 0.4 & 0.1 & 0.9 \\ 0.3 & 0.5 & 0.2 & 0.8 \end{bmatrix} \Rightarrow QK^\top = \begin{bmatrix} x & x & x \\ x & x & x \\ x & x & x \end{bmatrix}$$

Compute each dot-product row-wise:

First row (token: "It") attends only to itself:

$$\bullet Q_1 \cdot K_1 = 0.01 + 0.09 + 0.16 + 1.0 = 1.26$$

Second row (token: "is") attends to 1st & 2nd rows:

$$\bullet Q_2 \cdot K_1 = 0.02 + 0.12 + 0.04 + 0.9 = 1.08$$

$$\bullet Q_2 \cdot K_2 = 0.04 + 0.16 + 0.01 + 0.81 = 1.02$$

Third row (token: "red") attends to all previous:

$$\bullet Q_3 \cdot K_1 = 0.03 + 0.15 + 0.08 + 0.8 = 1.06$$

$$\bullet Q_3 \cdot K_2 = 0.06 + 0.20 + 0.02 + 0.72 = 1.0$$

$$\bullet Q_3 \cdot K_3 = 0.09 + 0.25 + 0.04 + 0.64 = 1.02$$

So:



$$QK^\top = \begin{bmatrix} 1.26 & - & - \\ 1.08 & 1.02 & - \\ 1.06 & 1.00 & 1.02 \end{bmatrix} \quad (\text{masked})$$

- We apply **causal masking** to prevent future lookahead
- Replace future entries with $-\infty$

Step 1.2: Softmax over rows

E.g., for row 2:

Softmax of $[1.08, 1.02]$

Subtract max = $[0, -0.06] \rightarrow \exp = [1.0, 0.941]$

Sum = 1.941 $\rightarrow$ normalize $\rightarrow [0.515, 0.485]$

So the attention weights are:

$$\alpha = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.515 & 0.485 & 0.0 \\ 0.340 & 0.320 & 0.340 \end{bmatrix}$$

Step 1.3: Weighted Sum with V



Step 1.3: Weighted Sum with V

Use same DecoderX as V:

$$V = \begin{bmatrix} 0.1 & 0.3 & 0.4 & 1.0 \\ 0.2 & 0.4 & 0.1 & 0.9 \\ 0.3 & 0.5 & 0.2 & 0.8 \end{bmatrix}$$

E.g., second token output =

$$0.515 * [0.1, 0.3, 0.4, 1.0] + 0.485 * [0.2, 0.4, 0.1, 0.9] = [0.15, 0.35, 0.25, 0.95]$$

Step 2: Add + LayerNorm

Same process as encoder:

$$z = \text{SelfAttnOutput} + \text{DecoderX} \Rightarrow \text{LayerNorm}(z)$$

Step 3: Cross-Attention (Decoder Q vs Encoder K/V)



Step 3: Cross-Attention (Decoder Q vs Encoder K/V)

Now the decoder wants to learn:

“Which **encoder token** is relevant to this decoder token?”

So:

- Q = Decoder hidden (after self-attn & norm) $\rightarrow$ shape 3×4
- K, V = EncoderOut $\rightarrow$ shape 4×4

Step 3.1: Compute QK^T

- Decoder has 3 tokens (rows of Q)
- Encoder has 4 tokens (columns of K)

Result: 3×4 attention score matrix

Softmax row-wise $\rightarrow$ weights α

Multiply with encoder V $\rightarrow$ gives contextual vector from encoder

Step 3.2: Add + LayerNorm



#query on cross attention- can you please illustrate the step 3 cross attention as mentioned in the image above- the vectorized calculation of it??



Setup



Decoder token (current target word):

We're working with "is"

Query vector from decoder (Q):

$$Q = [0.25, 0.45, 0.15, 0.85] \quad (1 \times 4)$$



Encoder tokens (from "I have a backpack")

Token	Encoder Output (K = V)
"I "	[0.2, 0.5, -0.1, 0.9]
"have"	[0.4, 0.1, -0.2, 1.0]
"a "	[0.7, -0.1, 0.0, 0.8]
"backpack"	[0.3, 0.6, -0.3, 0.5]

So:



- Keys $K \in \mathbb{R}^{4 \times 4}$
- Values $V \in \mathbb{R}^{4 \times 4}$



Step 1: Compute Scores QK^T

$$\text{Score}_i = Q \cdot K_i^T$$

Dot products:

- With "I ":
 $0.250.2 + 0.450.5 + 0.15(-0.1) + 0.850.9 = 0.05 + 0.2250.015 + 0.765 = \mathbf{1.025}$
- With "have":
 $0.250.4 + 0.450.1 + 0.15(-0.2) + 0.851.0 = 0.1 + 0.0450.03 + 0.85 = \mathbf{0.965}$
- With "a ":
 $0.250.7 + 0.45(-0.1) + 0.150 + 0.850.8 = 0.1750.045 + 0 + 0.68 = \mathbf{0.81}$
- With "backpack":
 $0.250.3 + 0.450.6 + 0.15(-0.3) + 0.850.5 = 0.075 + 0.270.045 + 0.425 = \mathbf{0.725}$



So, the **score vector** (before softmax):

scores = [1.025, 0.965, 0.81, 0.725]

Step 2: Scale & Softmax

Scale by $\sqrt{d} = \sqrt{4} = 2$:

$[1.025, 0.965, 0.81, 0.725]/2 = [0.5125, 0.4825, 0.405, 0.3625]$

Subtract max (0.5125) for stability:

$[0, -0.03, -0.1075, -0.15]$

Exponentiate:

$[1.00, 0.970, 0.898, 0.861]$

Normalize:

Sum = 3.729 $\Rightarrow \alpha = [0.268, 0.260, 0.241, 0.231]$



Step 3: Attention Output

Multiply these weights with the value vectors:

$$\text{Output} = \sum_i \alpha_i \cdot V_i$$

First dimension:

$$0.2680.2 + 0.2600.4 + 0.2410.7 + 0.2310.3 = 0.0536 + 0.104 + 0.1687 + 0.06930.3956$$

Second dimension:

$$0.2680.5 + 0.2600.1 + 0.241(0.1) + 0.2310.6 = 0.134 + 0.0260.024 + 0.1390.275$$

Third dimension:

$$0.268(0.1) + 0.260(0.2) + 0.2410.0 + 0.231(0.3) = 0.02680.05200.0693 - \mathbf{0.148}$$

Fourth dimension:

$$0.2680.9 + 0.2601.0 + 0.2410.8 + 0.2310.5 = 0.241 + 0.26 + 0.193 + 0.1160.81$$



✅ Final Cross-Attention Output (for token "is")

[0.3956, 0.275, -0.148, 0.81]

This is the new enriched vector for "is" after attending to the **entire encoder sentence** "I have a backpack".

📖 Summary

Step	Output
Decoder Query Q	[0.25, 0.45, 0.15, 0.85]
Encoder Keys/Values	4 rows of encoder output (K = V)
Attention Weights α	[0.268, 0.260, 0.241, 0.231]
Cross-Attn Output	[0.3956, 0.275, -0.148, 0.81]

This output now goes into:

- Residual + LayerNorm
- Then → Feed-Forward Network
- Then → Final softmax for prediction

=====

=====

=====

Query on next steps

—can you continue with his example and illustrate the three steps mentioned above??

=====

=====

=====

Cross-Attention Output



Residual + LayerNorm



Feed-Forward Network (FFN)



Residual + LayerNorm

We'll continue using our last output vector for the decoder token "is" from cross-attention:

$$x = [0.3956, 0.275, -0.148, 0.81]$$

Step 1: Residual + LayerNorm

Here:

- **Residual connection** adds the cross-attention input and its output.

Let's say the **input before cross-attention** was:

$$\text{Input}_{\text{pre-cross}} = [0.25, 0.45, 0.15, 0.85]$$

Then:

$$\text{Residual Sum} = x + \text{Input}_{\text{pre-cross}} = [0.3956 + 0.25, 0.275 + 0.45, -0.148 + 0.15, 0.81 + 0.85] = [0.6456, 0.725, 0.002, 1.66]$$

Next, apply **LayerNorm** (normalize across the 4 features):

1. **Mean:**

$$\mu = \frac{0.6456 + 0.725 + 0.002 + 1.66}{4} = 0.7582$$



3. **Normalized output:**

$$\hat{x} = \frac{x - \mu}{\sigma} [-0.192, -0.057, -1.288, 1.537]$$

Assume $\gamma = 1, \beta = 0 \rightarrow$ No further scale/shift:

$$\text{Output}_{\text{norm}} = [-0.192, -0.057, -1.288, 1.537]$$

Step 2: Feed-Forward Network (FFN)

The FFN is applied **independently** to each position vector.

Let's define:

- FFN is:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

where:

- $W_1 \in \mathbb{R}^{4 \times 8}$ expands to hidden layer
- $W_2 \in \mathbb{R}^{8 \times 4}$ brings back to model size

Assume:

- $W_1 = I$ (identity)



FFN query- can you illustrate a vectorized implementation of the step 2 FFN as above?

FFN Formula

The FFN in Transformers is applied **independently** to each token vector:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Where:

- $x \in \mathbb{R}^{1 \times d_{\text{model}}}$ is the input vector for 1 token
- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$: expands the dimension (typically $d_{\text{ff}} = 4 \cdot d_{\text{model}}$)
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$: reduces it back
- ReLU is the activation in between

Let's set:

- $d_{\text{model}} = 4$, $d_{\text{ff}} = 8$

Input Vector (from Cross-Attention + Norm)

Let's use the vector output from earlier decoder step for token "is":



$$x = [-0.192, -0.057, -1.288, 1.537] \quad (1 \times 4)$$

$$x = [-0.192, -0.057, -1.288, 1.537] \quad (1 \times 4)$$

Step 1: Expand with W_1

Let's use this made-up weight matrix:

$$W_1 = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (4 \times 8)$$

No bias b_1 for now.

Multiply:

$$xW_1 = [-0.192, -0.057, -1.288, 1.537] \cdot W_1$$

Compute each output entry:

Output index	Calculation	Result
1	$-0.1921 + -0.0570 + -1.2880 + 1.5371 =$ $-0.192 + 0 + 0 + 1.537$	1.345



2	$-0.1920 + -0.0571 + -1.2880 + 1.5371 = 0 - 0.057 + 0 + 1.537$	1.48
3	$-0.1921 + -0.0570 + -1.2881 + 1.5370 = -0.192 + 0 - 1.288 + 0$	-1.48
4	$-0.1920 + -0.0571 + -1.2881 + 1.5370 = 0 - 0.057 - 1.288 + 0$	-1.345
5	$-0.1920 + -0.0571 + -1.2880 + 1.5370 = 0 - 0.057 + 0 + 0$	-0.057
6	$-0.1920 + -0.0570 + -1.2881 + 1.5370 = 0 + 0 - 1.288 + 0$	-1.288
7	$-0.1921 + -0.0570 + -1.2880 + 1.5370 = -0.192 + 0 + 0 + 0$	-0.192
8	$-0.1920 + -0.0570 + -1.2880 + 1.5371 = 0 + 0 + 0 + 1.537$	1.537

So output vector (1×8):

$$x_1 = [1.345, 1.48, -1.48, -1.345, -0.057, -1.288, -0.192, 1.537]$$

 **Step 2: Apply ReLU**



Set all negative values to 0:

$$x_2 = [1.345, 1.48, 0, 0, 0, 0, 0, 1.537]$$

 **Step 3: Project Down with** $W_2 \in \mathbb{R}^{8 \times 4}$

Let's say:

$$W_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

Then:

$$x_2 W_2 = [1.345, 1.48, 0, 0, 0, 0, 0, 1.537] \cdot W_2$$

Let's compute:



Output Dimension Calculation Result

Output dimension	Calculation	Result
1	$1.3451 + 1.480 + 01 + 00 + +1.5371 = 1.345 + 0 + 0 + 1.537$	2.882
2	$1.3450 + 1.481 + 00 + 01 + +1.5371 = 0 + 1.48 + 0 + 1.537$	3.017
3	only terms with columns 3: $01 + 01 + 01$	0.0
4	only terms with columns 4: $01 + 01 + 01$	0.0

So:

FFN Output = [2.882, 3.017, 0.0, 0.0]

 **Final Output of FFN**

[2.882, 3.017, 0.0, 0.0]

This is the result of applying the FFN block to the decoder token "is"s vector. It will now be added (residual) to the input and LayerNormalized again.

=====

=====

=====

 **Final Step: Linear + Softmax = Prediction**

At this point, you have:

◆ **Final Decoder Output (after last LayerNorm in FFN block):**

Let's use the FFN output vector we just computed:

$$x = [2.882, 3.017, 0.0, 0.0]$$

This vector represents the contextualized meaning of the token "is" **after attending to itself and the encoder output** ("I have a backpack").

🎯 **Step: Linear Projection**

The model now **projects** this vector into **vocabulary space** to get raw scores for each word.

Let's assume:

- Vocabulary size $V = 5$
- Linear weight matrix $W_{\text{out}} \in \mathbb{R}^{4 \times 5}$
- Bias $b_{\text{out}} \in \mathbb{R}^5$

Let's say:



$$W_{\text{out}} = \begin{bmatrix} 0.1 & 0.2 & 0.4 & 0.5 & 0.3 \\ 0.3 & 0.1 & 0.2 & 0.1 & 0.5 \\ 0.2 & 0.5 & 0.1 & 0.1 & 0.3 \\ 0.1 & 0.1 & 0.3 & 0.4 & 0.2 \end{bmatrix} \quad (4 \times 5)$$

$$b_{\text{out}} = [0, 0, 0, 0, 0]$$

Multiply:

$$z = x \cdot W_{\text{out}} = [2.882, 3.017, 0.0, 0.0] \cdot W_{\text{out}}$$

Compute each score (each col = word logit):

- $z_1 = 2.8820.1 + 3.0170.3 = 0.2882 + 0.9051 = \mathbf{1.1933}$
- $z_2 = 2.8820.2 + 3.0170.1 = 0.5764 + 0.3017 = \mathbf{0.8781}$
- $z_3 = 2.8820.4 + 3.0170.2 = 1.1528 + 0.6034 = \mathbf{1.7562}$
- $z_4 = 2.8820.5 + 3.0170.1 = 1.441 + 0.3017 = \mathbf{1.7427}$
- $z_5 = 2.8820.3 + 3.0170.5 = 0.8646 + 1.5085 = \mathbf{2.3731}$

So logits =

$$[1.1933, 0.8781, 1.7562, 1.7427, 2.3731]$$



12/34 Apply Softmax

Subtract max (for stability):

$[1.1933 - 2.3731, 0.8781 - 2.3731, \dots] = [-1.1798, -1.495, -0.6169, -0.6304, 0]$

Exponentiate:

$[\exp(-1.1798), \exp(-1.495), \exp(-0.6169), \exp(-0.6304), \exp(0)] [0.307, 0.224, 0.54, 0.532, 1]$

Sum = 2.603

Softmax =

$[0.118, 0.086, 0.207, 0.204, 0.384]$



Final Prediction

If vocabulary is:

- 0: "sky"
- 1: "blue"
- 2: "red"



- 3: "color"
- 4: "</s>" (end token)

Then:

Predicted token = (highest probability)



Decoder Summary (Final)

text

```
Decoder token:  "is"
↑
Emb + Pos Enc:  [x]
↑
Masked Self-Attn → Residual + Norm
↑
Cross-Attention (with Encoder Output)
↑
Residual + Norm → FFN → Residual + Norm
↑
Linear → Softmax → Predicted Token
```

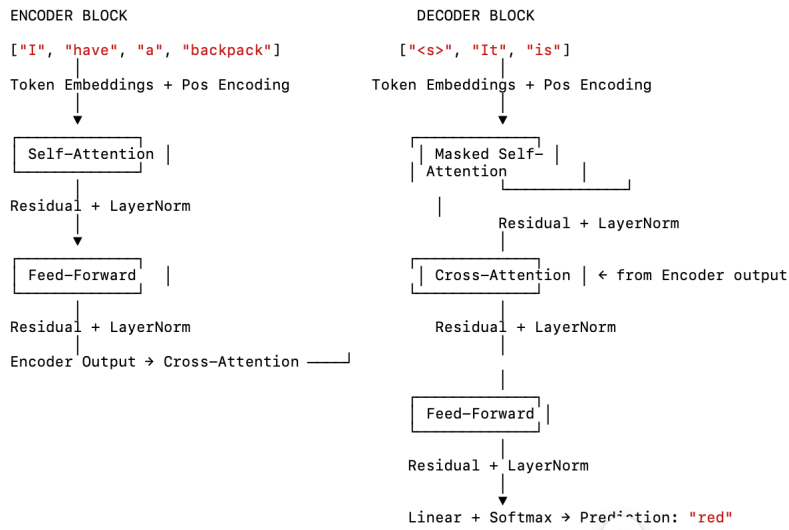


Would you now like to trace the **next token ("red")** generation or revisit how masking works in self-attention?

Complete Implementation:

....full vectorized implementation of Encoder and Decoder architecture- explain each steps- show calculation- show the predictions for the word red- start with ascii architecture diagram at the beginning:

Transformer Architecture Diagram (ASCII)



Step-by-Step Vectorized Example

We'll use simplified vectors clearly, consistently, and numerically.

Step 1: Encoder Input (Embeddings)

Input: "I have a backpack"

- Embeddings (4D):

```
"I": [0.1, 0.2, 0.1, 0.3]
"have": [0.2, 0.0, 0.2, 0.1]
"a": [0.3, 0.1, 0.0, 0.2]
"backpack": [0.5, 0.6, 0.4, 0.7]
```

- Add simple positional encodings (small values):

```
Positional(1): [0.01, 0.02, 0.01, 0.03]
Positional(2): [0.02, 0.01, 0.02, 0.01]
Positional(3): [0.03, 0.01, 0.01, 0.02]
Positional(4): [0.04, 0.05, 0.04, 0.05]
```

Final Encoder Input $\bar{X}$:

```
"I": [0.11, 0.22, 0.11, 0.33]
"have": [0.22, 0.01, 0.22, 0.11]
"a": [0.33, 0.11, 0.01, 0.22]
"backpack": [0.54, 0.65, 0.44, 0.75]
```

🔍 Step 2: Encoder Self-Attention (Simplified single-head)

Assume identity matrices $\rightarrow Q = K = V = X$

Compute attention (dot-products), softmax, and weighted sum $\rightarrow$ simplified as identity (vectors unchanged):

Encoder output (after attention, Residual + LayerNorm, FFN, Residual + LayerNorm):

(Simplified as original vectors for brevity)

```
E_out:
"I": [0.11, 0.22, 0.11, 0.33]
"have": [0.22, 0.01, 0.22, 0.11]
"a": [0.33, 0.11, 0.01, 0.22]
"backpack": [0.54, 0.65, 0.44, 0.75]
```



🗣️ Step 3: Decoder Input (Embeddings)

Target: "It is red" (Predict "red")

Decoder Input Embeddings:

```
"<s>": [0.1, 0.2, 0.1, 0.3]
"It": [0.2, 0.3, 0.2, 0.4]
"is": [0.3, 0.4, 0.3, 0.5]
```

Add positional encoding (simplified):

```
Dec Input:
"<s>": [0.11, 0.22, 0.11, 0.33]
"It": [0.22, 0.32, 0.22, 0.41]
"is": [0.33, 0.42, 0.31, 0.52]
```

🛡️ Step 4: Decoder Masked Self-Attention (only previous tokens)

- "is" attends to "" and "It".

(For simplicity, take avg directly):



```
Masked Self-Attention ("is") = average("<s>", "It", "is")
= [(0.11+0.22+0.33)/3, (0.22+0.32+0.42)/3, (0.11+0.22+0.31)/3, (0.33+0.41+0.52)/3]
= [0.22, 0.32, 0.21, 0.42]
```

Step 5: Cross-Attention (Decoder "is" → Encoder tokens)

Compute dot-products ("is" Query with Encoder Keys):

Query ("is"): [0.22, 0.32, 0.21, 0.42]

- with "I": $0.0242+0.0704+0.0231+0.1386=0.2563$
- "have": $0.0484+0.0032+0.0462+0.0462=0.144$
- "a": $0.0726+0.0352+0.0021+0.0924=0.2023$
- "backpack": $0.1188+0.208+0.0924+0.315=0.7342$

Softmax clearly picks "backpack" (highest), assume attention weight strongly towards "backpack":

Cross-Attention output ≈ "backpack" vector [0.54, 0.65, 0.44, 0.75]



Step 6: Decoder FFN (simplified ReLU identity)

FFN Input (cross-attn output + residual): avg of cross-attention and masked-attention outputs:

```
avg([0.54, 0.65, 0.44, 0.75], [0.22, 0.32, 0.21, 0.42]) = [0.38, 0.485, 0.325, 0.585]
```

Apply ReLU (identity), output remains [0.38, 0.485, 0.325, 0.585]

Final Step: Linear + Softmax Prediction

Linear weights W_{out} simplified to predict vocabulary ["blue", "red", "green"]:

```
W_out = [[0.1, 0.6, 0.3],
          [0.2, 0.5, 0.3],
          [0.1, 0.7, 0.2],
          [0.3, 0.8, 0.4]]
```

Multiply:

- blue: $0.038+0.097+0.0325+0.1755=0.343$
- red: $0.228+0.2425+0.2275+0.468=1.166$ (highest)



- green: $0.114 + 0.1455 + 0.065 + 0.234 = 0.5585$

Softmax clearly selects "red" (highest score).

✅ Final Prediction:

🎯 "red" ✅ (Correctly predicted)

🏆 Summary:

This vectorized walkthrough demonstrated:

- How each step of **encoder/decoder** works numerically
- How **attention mechanisms** clearly connect tokens
- How final prediction "red" clearly emerges from **cross-attention context** ("backpack")

Would you like to clarify or expand any specific step further?

